WiFi-CSI HAR: Complete Code Architecture & Coding Viva Guide

Line-by-Line Code Walkthrough, PyTorch Tensor Pipeline, & 25+ Code-Focused Viva Questions

Target Codebase: `src/models.py`, `src/dataset.py`, `src/train.py`, `src/evaluate.py`, `src/live_inference.py`, `live_demo.py`	**Deep Learning Framework:** PyTorch 2.x, NumPy, scikit-learn
Key Architectures: Baseline CNN-BiLSTM, MS-CNN-BiGRU, Champion SE-TCN (110k params)	**Academic Context:** ECE Dept, RNSIT (Group 10, 2026-2027)

1. Deep-Dive: Model Architectures (`src/models.py`)

The model file implements 7 architectures: Baselines (MLP, LeNet, BiLSTM, CNN-GRU, CNN-BiLSTM) and two proposed novel architectures (MS-CNN-BiGRU and SE-TCN). Understanding the tensor shapes across every layer is critical for code viva defense.

A. Tensor Dimensions & The Crucial Permutation Trick:

PyTorch layers expect different tensor channel orderings:
• **Dataset / DataLoader Output:** [Batch, 1, 250, 90] or [Batch, 250, 90] (B, Time=250, Features=90).
• **Conv1D Expects:** [Batch, Channels, Time] $\rightarrow$ Channels=90, Time=250.
• **LSTM / GRU Expects (with batch_first=True):** [Batch, Time, Channels] $\rightarrow$ Time=125, Features=128.
Therefore, in every model forward pass, you will see explicit permutes:
x = x.permute(0, 2, 1) # [B, 250, 90] -> [B, 90, 250] for Conv1D
x = self.encoder(x) # Conv1D feature extraction -> [B, 128, 125]
x = x.permute(0, 2, 1) # [B, 128, 125] -> [B, 125, 128] for LSTM/GRU

B. Champion Architecture: Squeeze-and-Excitation TCN (`SE_TCN`):

The champion model contains 4 core sub-components. Here is the exact implementation structure from src/models.py:

```
# 1. Spatial Projection & Channel Attention:
self.in_proj = nn.Sequential(nn.Conv1d(90, 64, kernel_size=1), nn.BatchNorm1d(64), nn.ReLU())
self.attn    = Subcarrier_SE_Attention(channels=64, reduction=4)

# 2. Four Stacked Residual Dilated TCN Blocks (Exponential Dilations d = [1, 2, 4, 8]):
self.layer1 = Residual_Dilated_Conv1D(64, kernel_size=3, dilation=1)
self.layer2 = Residual_Dilated_Conv1D(64, kernel_size=3, dilation=2)
self.layer3 = Residual_Dilated_Conv1D(64, kernel_size=3, dilation=4)
self.layer4 = Residual_Dilated_Conv1D(64, kernel_size=3, dilation=8)

# 3. Global Temporal Average Pooling & Linear Classifier:
self.global_pool = nn.AdaptiveAvgPool1d(1)  # [B, 64, 250] -> [B, 64, 1]
self.classifier  = nn.Sequential(nn.Dropout(0.2), nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 5))
```

C. How Dilated Residual Blocks Preserve Sequence Length:

In Residual_Dilated_Conv1D, to ensure that the output sequence length remains exactly equal to input length (250) so the residual skip connection out + res can add them without dimension mismatch, padding is calculated dynamically:
padding = (kernel_size - 1) * dilation // 2
For $k=3$: if $d=1 \rightarrow \text{pad}=1$; if $d=2 \rightarrow \text{pad}=2$; if $d=4 \rightarrow \text{pad}=4$; if $d=8 \rightarrow \text{pad}=8$. This guarantees identical length across all 4 stages while expanding the effective receptive field beyond 120 steps.

D. Proposed Model 2: Multi-Scale CNN-BiGRU (`MS_CNN_BiGRU`):

MultiScale_Conv1D_Block splits out_channels across three parallel conv branches with kernel sizes 3, 5, and 7:

- **Kernel 3 (padding 1):** Captures high-frequency, abrupt motion transients (sudden falls, rapid limb jerks).
- **Kernel 5 (padding 2):** Captures intermediate cyclic motions (rhythmic walking gait cadence).
- **Kernel 7 (padding 3):** Captures slow, prolonged postural transitions (sitting down, standing up).

The branches are concatenated along the channel dimension: torch.cat([b1(x), b2(x), b3(x)], dim=1). A BiGRU follows: torch.cat((ht[-2], ht[-1]), dim=1) concatenates the forward and backward final hidden states into a 192-dim vector.

2. Deep-Dive: Preprocessing & Data Leakage Prevention (`src/dataset.py`)

A. Raw CSI Subcarrier Interpolation (`raw_csi_to_features`):

Each raw ESP32 packet reports 384 integer values representing 192 interleaved (In-Phase, Quadrature) pairs. The feature extraction logic is:

```
real = arr[0::2]                          # 192 In-phase elements
imag = arr[1::2]                          # 192 Quadrature elements
mags = np.sqrt(real**2 + imag**2)         # Subcarrier amplitudes
nonzero = mags[mags > 0]                  # Filter guard band / null subcarriers (~108 active)
old_x = np.linspace(0, 1, len(nonzero))
new_x = np.linspace(0, 1, target_features) # Exactly 90 uniform points
features = np.interp(new_x, old_x, nonzero).astype(np.float32)  # Linear interpolation
```

B. Sliding Window Slicing (`make_windows_from_recording`):

Consecutive packet streams are converted into tensor windows of shape [N, 250, 90] using stride 50:
starts = range(0, len(frames) - window_size + 1, stride)
windows = np.stack([frames[s : s + window_size] for s in starts])
At 100 Hz sampling rate, 250 packets = 2.50 seconds. A stride of 50 packets = 0.50 seconds advance (80% overlap).

C. The Leakage-Free Grouped Split Implementation (`build_grouped_dataset`):

To prevent overlapping windows from leaking between sets, the split is performed at the **recording session level** rather than window level:

```
# 1. Load all recordings grouped by class and filename
for c in CLASS_NAMES:
    recs = list(by_class[c])
    rng.shuffle(recs)                     # Shuffle whole recording files, NOT windows
    n_train = int(round(n_recs * 0.70))   # 70% whole recordings to train
    n_val   = int(round(n_recs * 0.15))   # 15% whole recordings to val
    n_test  = n_recs - n_train - n_val    # 15% whole recordings strictly to test

# 2. Fit normalizer STRICTLY on training split
tr_min = float(X_train_raw.min())
tr_max = float(X_train_raw.max())
X_train = (X_train_raw - tr_min) / (tr_max - tr_min)
X_val   = (X_val_raw   - tr_min) / (tr_max - tr_min)  # Scaled using TRAIN stats
X_test  = (X_test_raw  - tr_min) / (tr_max - tr_min)  # Zero test statistics leakage
```

3. Deep-Dive: Training Loop, Focal Loss & Validation (`src/train.py`)

A. Multiclass Focal Loss Implementation (`FocalLoss`):

Standard Cross-Entropy loss is overwhelmed by majority classes (Empty, Walking). Focal Loss downweights easy examples using $(1 - p_t)^\gamma$:

```
class FocalLoss(nn.Module):
    def __init__(self, alpha=None, gamma=2.0):
        super().__init__()
        self.alpha = alpha  # Optional per-class balancing weight tensor [5]
        self.gamma = gamma  # Focusing parameter gamma=2.0

    def forward(self, logits: torch.Tensor, targets: torch.Tensor) -> torch.Tensor:
        # reduction='none' keeps unreduced per-sample loss [Batch]
        ce_loss = F.cross_entropy(logits, targets, reduction='none', weight=self.alpha)
        pt = torch.exp(-ce_loss)  # p_t is the model's estimated probability for true class
        focal_loss = ((1.0 - pt) ** self.gamma) * ce_loss
        return focal_loss.mean()
```

B. The 5-Step PyTorch Training Step (`train_epoch`):

Inside the batch training loop, 5 essential operations execute in strict order:
1. optimizer.zero_grad(): Clears accumulated gradients from previous batch.
2. outputs = model(inputs): Forward pass computing raw unconstrained logits [B, 5].
3. loss = criterion(outputs, labels): Computes scalar loss via Cross-Entropy or Focal Loss.
4. loss.backward(): Backpropagates loss through autograd graph, calculating $\partial L / \partial w$.
5. optimizer.step(): Updates weights using Adam optimizer: $w_{t+1} = w_t - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$.

C. Validation & Early Stopping on Macro-F1:

Validation runs with model.eval() and with torch.no_grad():. Crucially, early stopping monitors **Validation Macro-F1** rather than raw validation loss or accuracy. If a model predicts Empty for everything, accuracy would be 35% with low loss, but Macro-F1 would be near 0.10. Monitoring Macro-F1 forces the model to learn all classes, including rare falls.

4. Deep-Dive: Live Streaming Inference Engine (`src/live_inference.py`)

A. Circular FIFO Ring Buffer Implementation:

Real-time continuous inference requires maintaining a rolling history of exactly 250 packets without allocating new arrays every 10 ms. This is implemented using Python's double-ended queue (collections.deque):
self.packet_buffer = deque(maxlen=window_size) # maxlen=250
When a new packet arrives via packet_buffer.append(feat), if the buffer has 250 packets, the oldest packet is evicted automatically in **O(1) time**. Standard Python lists require $O(N)$ memory shifts upon eviction; `deque` provides zero-copy ring buffer performance.

B. Dual-Stage Fall Detection Safety Threshold:

Standard classification uses argmax: predicted_class = argmax(probabilities). However, in eldercare, a missed fall is unacceptable. In live_inference.py, we implement an asymmetric safety threshold:

```
probs = F.softmax(logits, dim=1).cpu().numpy()[0]
fall_idx = CLASS_TO_IDX['Falling']  # Index 4
fall_prob = float(probs[fall_idx])

# Asymmetric Safety Rule: Trigger fall alert if P(Fall) > 0.35
if fall_prob >= self.fall_threshold:  # default 0.35
    alert_triggered = True
    final_class = 'Falling'
else:
    alert_triggered = False
    final_class = self.classes[int(np.argmax(probs))]
```

5. 25+ Dedicated Code & PyTorch Viva Examination Questions & Answers

These questions test whether you wrote, understand, and debugged the code yourself. Memorize the exact rationale for each answer.

CQ1: Why do we write `x.permute(0, 2, 1)` instead of `x.view()` or `x.reshape()` in model forward passes?

Model Answer: `view()` and `reshape()` can only change tensor shapes if the underlying memory layout is contiguous, and they do NOT transpose data elements across memory strides. Using `view(B, 90, 250)` on a `[B, 250, 90]` tensor would corrupt the data by reading the first 90 values of time step 0 as channel 0 across all time steps!
`permute(0, 2, 1)` explicitly transposes axes 1 and 2, swapping the Time and Channel dimensions correctly so that Conv1d filters slide along the temporal axis. If `view()` is subsequently needed after `permute()`, you must call `.contiguous().view()` because transposing makes the tensor non-contiguous in RAM.

Key Coding Takeaway: `permute` physically swaps axes; `view` merely reinterprets memory and would scramble CSI data.

CQ2: What happens if you forget to call `optimizer.zero_grad()` before `loss.backward()`?

Model Answer: In PyTorch, gradients accumulate by default: every call to `loss.backward()` adds (via `+=`) newly computed partial derivatives $\partial L / \partial w$ into `param.grad`. If you omit `optimizer.zero_grad()`, gradients from the current batch are added to gradients from all preceding batches, resulting in exploded gradient updates and unstable training divergence.

Key Coding Takeaway: PyTorch accumulates gradients by design; omitting `zero_grad()` causes gradient explosion.

CQ3: What does `model.train()` vs `model.eval()` actually do under the hood?

Model Answer: They do NOT freeze weights. Instead, they toggle the operational behavior of two specific PyTorch layer types:
1. **Dropout (`nn.Dropout`):** In `model.train()`, neurons are randomly zeroed with probability p. In `model.eval()`, dropout is disabled and all activations pass through scaled by 1.0.
2. **BatchNorm (`nn.BatchNorm1d` / `2d`):** In `train()`, it normalizes activations using the current mini-batch mean μ_B and variance σ_B^2, updating running statistics via momentum. In `eval()`, it freezes running statistics and normalizes using the accumulated population mean μ_{run} and σ_{run}^2. Evaluating in train mode causes wild prediction instability on small batches.

Key Coding Takeaway: `model.eval()` freezes BatchNorm running statistics and disables Dropout randomness.

CQ4: Why is validation wrapped in `with torch.no_grad():`?

Model Answer: During forward passes, PyTorch automatically constructs a dynamic computation graph (autograd tape) tracking intermediate activation tensors to compute gradients during backprop. Wrapping code in `with torch.no_grad():` disables autograd graph construction, saving ~60% GPU/CPU RAM and speeding up validation inference by 2× to 3×.

Key Coding Takeaway: `torch.no_grad()` prevents computation graph creation, cutting RAM usage and boosting speed.

CQ5: Why do we save `model.state_dict()` instead of saving the whole model object `torch.save(model)`?

Model Answer: Saving the entire object `torch.save(model, path)` uses Python's `pickle` module, which binds the saved file to the exact directory layout, source code file paths, and Python class definitions. If you rename a class or move files, unpickling fails. Saving `model.state_dict()` saves only a clean dictionary mapping parameter names to raw weight tensors (e.g., `'layer1.conv1.weight': tensor(...)`). It is lightweight, robust to code refactoring, and compatible across different PyTorch versions.

Key Coding Takeaway: `state_dict` saves pure numerical weights; whole-model pickling is brittle and breaks on code changes.

CQ6: In `SE_TCN`, explain the padding formula: `padding = (kernel_size - 1) * dilation // 2`.

Model Answer: The output sequence length of a 1D convolution is given by: $L_{out} = \lfloor \frac{L_{in} + 2P - d(K - 1) - 1}{S} + 1 \rfloor$.
For stride $S = 1$, setting $L_{out} = L_{in}$ requires: $2P = d(K - 1) \implies P = \frac{d(K - 1)}{2}$. For kernel size $K = 3$, $K - 1 = 2$, so $P = d$. Thus, setting `padding = dilation` guarantees that input length 250 remains exactly 250, allowing element-wise residual addition `out + res` without interpolation.

Key Coding Takeaway: Padding equals dilation factor for $K=3$, keeping tensor length constant for residual addition.

CQ7: In `FocalLoss`, why is `reduction='none'` used inside `F.cross_entropy`?

Model Answer: By default, `F.cross_entropy` averages loss across the entire batch, returning a single scalar. However, Focal Loss needs to compute a unique modulating weight $(1 - p_{t,i})^\gamma$ for *each individual sample* i in the batch. Setting `reduction='none'` returns a 1D tensor of losses `[Batch]`, allowing element-wise multiplication by $((1 - p_t)^\gamma)$ before taking `.mean()` across the batch.

Key Coding Takeaway: `reduction='none'` preserves individual sample losses to apply per-sample focal weighting.

CQ8: How does `pt = torch.exp(-ce_loss)` mathematically extract the true-class probability?

Model Answer: Cross-entropy loss for a single sample is mathematically defined as: $\text{CE} = -\log(p_t)$, where p_t is the softmax probability assigned to the ground-truth class. Taking the natural exponential: $\exp(-\text{CE}) = \exp(-(-\log(p_t))) = p_t$. This PyTorch trick avoids calling `F.softmax` and indexing into target classes, making the computation numerically stable and fast.

Key Coding Takeaway: `exp(-ce_loss)` directly recovers p_t because cross-entropy is $-\log(p_t)$.

CQ9: How does `torch.cat((ht[-2], ht[-1]), dim=1)` work in bidirectional LSTM / GRU?

Model Answer: In PyTorch, the hidden state tensor `ht` from a bidirectional RNN has shape `[num_layers * 2, Batch, hidden_dim]`. For a 1-layer bidirectional network, `ht[0]` contains the final hidden state of the forward RNN (processing $t=0 \rightarrow T$), and `ht[1]` (or `ht[-1]`) contains the final hidden state of the backward RNN (processing $t=T \rightarrow 0$). `torch.cat((ht[-2], ht[-1]), dim=1)` concatenates forward and reverse temporal context into a single descriptor `[Batch, hidden_dim * 2]`.

Key Coding Takeaway: `ht[-2]` is the forward RNN summary; `ht[-1]` is the backward RNN summary. Concatenating yields both.

CQ10: Why use `collections.deque(maxlen=250)` for the live FIFO buffer instead of a Python list?

Model Answer: A standard Python list has dynamic contiguous memory. Appending to a full list or popping from the left (`list.pop(0)`) requires shifting all 249 remaining elements in memory, incurring an $O(N)$ time penalty for every packet. `collections.deque` is implemented in C as a doubly linked list of fixed-size blocks. Appending and evicting from either end is strictly **O(1) constant time**, guaranteeing zero packet dropping at 100 Hz.

Key Coding Takeaway: `deque` provides $O(1)$ constant-time push/evict, avoiding list memory reallocation bottlenecks.

CQ11: Why is `num_workers=0` used in `DataLoader` on Windows?

Model Answer: On Windows, Python implements multiprocessing via `spawn` rather than `fork` (used in Linux). Spawning child processes requires serializing (`pickling`) the entire script context and re-importing main modules, which frequently causes multiprocessing deadlocks, file handle locking errors on `.npy` files, and high CPU startup overhead. Setting `num_workers=0` runs data loading synchronously in the main thread with zero IPC overhead.

Key Coding Takeaway: `num_workers=0` avoids Windows process spawning locks and pickling errors.

CQ12: In `CSIDataset`, why do we do `.unsqueeze(1)` on the input array?

Model Answer: `X` has raw shape `[N, 250, 90]`. Calling `.unsqueeze(1)` inserts a singleton channel dimension at axis 1, transforming it to `[N, 1, 250, 90]`. This conforms to standard PyTorch image/spectrogram conventions (`[Batch, Channel, Height, Width]`), ensuring compatibility with 2D CNN baselines (like LeNet) as well as 1D models.

Key Coding Takeaway: `.unsqueeze(1)` adds a channel dimension to create standard 4D PyTorch tensor format.

CQ13: How does `AdaptiveAvgPool1d(1)` eliminate sequence length constraints?

Model Answer: Standard pooling (`AvgPool1d(kernel_size=K)`) downsamples by a fixed ratio, producing output lengths that depend directly on input sequence length. `AdaptiveAvgPool1d(1)` dynamically computes the pooling kernel size to guarantee that the output temporal dimension is always exactly 1, regardless of whether input sequence length is 250, 100, or 500. It computes a global temporal average descriptor and removes positional bias.

Key Coding Takeaway: `AdaptiveAvgPool1d(1)` forces the temporal dimension to 1, enabling variable-length input flexibility.

CQ14: Why do we call `loss.item()` when tracking training loss?

Model Answer: `loss` is a PyTorch tensor attached to the dynamic autograd computation graph. If you write `total_loss += loss`, Python keeps the entire computation graph for that batch in RAM/VRAM across the entire epoch, leading to an **out-of-memory (OOM) memory leak**! Calling `loss.item()` extracts the pure Python `float` primitive, allowing PyTorch to free the computation graph immediately.

Key Coding Takeaway: `loss.item()` extracts a Python scalar, preventing memory leaks from accumulating autograd graphs.

CQ15: How would you export this PyTorch model to ONNX for edge deployment?

Model Answer: To export the champion SE-TCN to ONNX, you write:
dummy_input = torch.randn(1, 250, 90)
torch.onnx.export(model, dummy_input, 'se_tcn.onnx', input_names=['csi_window'], output_names=['logits'], dynamic_axes={'csi_window': {0: 'batch'}})
This serializes the model into an open standard graph format compatible with OpenVINO, TensorRT, and ESP-DL for microcontroller deployment.

Key Coding Takeaway: `torch.onnx.export` traces the PyTorch model graph with a dummy tensor into an edge-deployable `.onnx` file.